

Article

Pervasive Auto-Scaling Method for Improving the Quality of Resource Allocation in Cloud Platforms

Vimal Raja Rajasekar * and G. Santhi

Department of Information Technology, Puducherry Technological University, Puducherry 605014, India;
shanthikarthikeyan@ptuniv.edu.in

* Correspondence: vimalraja.r@ptuniv.edu.in

Abstract

Cloud resource provider deployment at random locations increases operational costs regardless of the application demand intervals. To provide adaptable load balancing under varying application traffic intervals, the auto-scaling concept has been introduced. This article introduces a Pervasive Auto-Scaling Method (PASM) for Computing Resource Allocation (CRA) to improve the application quality of service. In this auto-scaling method, deep reinforcement learning is employed to verify shared instances of up-scaling and down-scaling pervasively. The overflowing application demands are computed for their service failures and are used to train the learning network. In this process, the scaling is decided based on the maximum computing resource allocation to the demand ratio. Therefore, the learning network is also trained using scaling rates from the previous (completed) allocation intervals. This process is thus recurrent until maximum resource allocation with high sharing is achieved. The resource provider migrates to reduce the wait time based on the high-to-low demand ratio between successive computing intervals. This enhances the resource allocation rate without high wait times. The proposed method's performance is validated using the metrics resource allocation rate, service delay, allocated wait time, allocation failures, and resource utilization.



Academic Editor: Fabrizio Messina

Received: 28 June 2025

Revised: 27 September 2025

Accepted: 8 October 2025

Published: 18 November 2025

Citation: Rajasekar, V.R.; Santhi, G. Pervasive Auto-Scaling Method for Improving the Quality of Resource Allocation in Cloud Platforms. *Big Data Cogn. Comput.* **2025**, *9*, 294. <https://doi.org/10.3390/bdcc9110294>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

Keywords: auto-scaling; cloud computing; quality of service; reinforcement learning; resource allocation

1. Introduction

Load balancing is a process that distributes network traffic equally among applications. Load balancing is an important task to perform in every application, which improves the satisfaction range of users over demands [1]. Load balancing provides necessary resource provisioning for the cloud environment. Various load-balancing methods and techniques are used to reduce the computational cost of the applications [2]. An optimization technique is used for resource provisioning in clouds. The optimization technique identifies the unutilized resources that are present in the network and provides optimal services to perform for the users [3]. The optimization technique is mainly used in the cloud to improve the overall quality of service (QoS) for a range of applications [4]. A virtual machine (VM)-based load-balancing approach using a hybrid scheduling algorithm is commonly used in cloud platforms. The scheduling algorithm analyzes the resource necessity to perform tasks in the systems [5]. The analyzed data produces feasible data for load-balancing planning, which reduces the latency in the execution process. The VM-based approach increases the resource utilization ratio via load balancing [6].

Auto-scaling in cloud computing is a method that is used to adjust computational resources based on workload. Auto-scaling provides optimal resource utilization services to the network, which minimizes the energy consumption in performing microservices for the applications [7]. Auto-scaling-based load-balancing techniques are used in the cloud to enhance the feasibility range in the resource computing process. A virtual cluster architecture based on auto-scaling is used in the cloud for load balancing [8]. Cluster architecture virtualizes the resources that are required for the load-balancing process. The actual scalability and availability level of the resources are identified, which produces feasible services for the balancing process [9]. The cluster architecture provides proper balancing services to the servers, which improves the performance level of the cloud environments. An auto-scaling algorithm-based load-balancing technique is also used for cloud computing systems [10]. The auto-scaling algorithm analyzes the workload of the environment and provides optimal resource provisioning services to networks. The auto-scaling algorithm reduces the latency in performing tasks in clouds [11].

Machine learning (ML) algorithms are mostly used to improve the accuracy of the prediction process. ML algorithm-based auto-scaling methods are used in cloud environments [12]. The actual goal is to increase efficiency and reduce the computational cost of the applications. Long short-term memory (LSTM) algorithm-based auto-scaling is used in cloud environments [13]. The LSTM algorithm predicts the workload of the network for the resource allocation and load-balancing processes. The predicted information provides feasible details that eliminate unwanted delays in the auto-scaling process [14]. The LSTM algorithm increases the accuracy level in auto-scaling, which minimizes the computational latency in cloud networks. The Reinforcement Learning (RL)-enabled technique identifies the exact resource requirements and analyzes the workload of the process. The identified information provides the necessary data for the auto-scaling process. The RL-based technique improves the overall performance and reliability range of cloud environments [15,16].

1.1. Contributions

- The design, description, and analysis of the pervasive auto-scaling method for effective resource allocation in the cloud.
- The augmentation and sharing feature improvement of the resources using deep reinforcement learning to reduce failures.
- The simulation analysis of the proposed method using different metrics and methods to verify its efficiency.

1.2. Organization

The article's organization is as follows: Section 2 describes the related works proposed by various authors from the past related to autoscaling and cloud-based resource allocations. In Section 3, the proposed PSAM is described with detailed descriptions, illustrations, and mathematical models. Section 4 describes the comparative assessment of the proposed method using specific methods and metrics, followed by the conclusion and findings in Section 5.

2. Related Works

Khaleq et al. [17] developed an intelligent autonomous autoscaling system for microservices in cloud-based applications. The developed system is mainly used to identify the right values that are necessary for the scaling process. A reinforcement learning (RL) algorithm is used in the system, which provides optimal information for the autoscaling process. The developed system enhances the quality of service (QoS) range of applications.

Desmouceaux et al. [18] proposed a joint monitor-less load balancing and autoscaling method for data centers. Numerous autoscalers are used in the method, which analyzes and adjusts the load patterns in the centers. A segment routing technique is implemented here, which schedules the routes based on the loads. It increases the quality range of the service in data centers. The proposed method reduces the operational overhead and computational cost of performing tasks.

Iqbal et al. [19] designed a performance-varying virtual machine (VM) based autoscaling prediction method for multi-tier applications. The actual goal of the method is to address the challenges that are faced by the scalers. Minimal resources are used in the method, providing the necessary datasets for autoscaling. The designed method improves the accuracy ratio in autoscaling, which enhances the feasibility range of the systems.

Al Qassem et al. [20] introduced a proactive random forest (RF) autoscaler model for resource allocation in microservices. The actual goal is to guide the allocation process based on the availability of the resources. The exact location and workloads of the resources are predicted, which minimizes the latency in the computation process. The introduced model increases the accuracy and feasibility range of the resource allocation process.

Zeydan et al. [21] developed a multi-attribute decision-making (MADM) algorithm for scaling in virtual network functions (VNF). A clustering technique is also used here to manage the point of presence (PoP), which produces the necessary information for the auto-scaling process. It is mainly used to solve the problems that separate the scaling factors in microservices. The developed algorithm improves the overall performance by providing optimal scaling services.

Bento et al. [22] proposed a cost-availability-aware scaling (CAAS) approach for cloud services. The main aim of the approach is to evaluate the availability and scalability level of the applications. The CAAS approach is mainly used to satisfy the demand range of the users. Experimental results show that the proposed CAAS approach improves the operational functions of the applications.

Jena et al. [23] designed a heuristic algorithm-based autoscaling method for intercloud computing. The designed method identifies the issues that cause damage to the scaling process. It categorizes the resources based on the demands of the tasks and provides relevant scaling services to the application. It minimizes the resource utilization complexity and increases the speed ratio of the systems. The designed method improves the performance and significance level of the cloud computing systems.

Feng et al. [24] introduced an auto-scalable and fault-tolerant load-balancing mechanism for cloud computing. It is used as a dynamic load-balancing strategy, which creates an effective platform for the scaling process. The actual goal is to reduce the resource consumption range and to improve the efficiency level of the network. Experimental results show increases in the auto-scalability range of the applications.

Verma et al. [25] proposed efficient auto-scaling for host load prediction in VM migration. Auto-scaling is mainly used in the cloud, which provides proper virtualization in a computing system. The proposed method detects the exact host load of the system, which reduces the latency range in performing computational tasks. The proposed method maximizes the accuracy ratio in the host load prediction process.

Rout et al. [26] developed a new dynamic, scalable auto-scaling model for a cloud computing environment. The developed model is mostly used to enhance the scalability of the scaling process in applications. It is also used as a load balancer, which provides resources based on the load and priorities of the tasks. When compared with other models, the developed model increases the efficiency range of the cloud environments.

Sharvani et al. [27] introduced an auto-scaling approach for load balancing in the cloud environment. The introduced approach identifies the exact demands and availability

of the resources. The identified details produce feasible information for the load-balancing process. Important features and patterns are detected, which decreases the computational complexity in auto-scaling. The introduced approach balances the workload and demands of the application, which enhances the feasibility level of the targets.

Llorens-Carrodegua et al. [28] designed a software-defined networking (SDN) based solution for auto-scaling and load balancing in virtualized network function (VNF). The SDN controller analyzes the OpenFlow switches (OFS), which are required for redirecting the traffic in auto-scaling. The OFSs are managed and maintained via SDN, which produces the necessary information for the load-balancing process. Experimental results show that the designed model reduces the overall traffic in resource allocation and load balancing processes.

Chouliaras et al. [29] proposed an auto-scaling framework for cloud resource management. The actual aim of the framework is to provide reliable resource provisioning in the cloud environment. The proposed framework uses a convolutional neural network (CNN) algorithm to extract the optimal dataset for the scaling process. The proposed framework provides the requirements for the process that improves the performance range of the networks.

Kim et al. [30] introduced a new cloud workload prediction framework, named Cloud Insight, for resource management prediction. It is used as a prediction framework that predicts the actual workload and resource availability range for cloud applications. An ensemble model is implemented here, which identifies the necessity and scalability ratio of the resources that minimizes the workload of the systems. The introduced framework increases the accuracy level in the prediction process.

Adewojo et al. [31] developed a weight-assignment load-balancing algorithm for cloud-enabled applications. The developed algorithm measures the resource usability and availability in the cloud for the resource allocation process. It also minimizes the response time latency, which enhances the feasibility ratio of the load-balancing process. The developed algorithm reduces the overall resource failure range in cloud environments.

The auto-scaling problem in the cloud is tedious due to erratic user/service demands under distinguishable intervals. Virtual machine migrations are reliant on limited demands to satisfy service allocations. Such processes are subject to availability and response as discussed in the methods above. A few other methods follow workload distribution and load balancing or inter-cloud sharing to achieve balanced resource sharing. This process is terminated/confined through periodic interval variations, and this increases the wait time. To address such issues, a pervasive auto-scaling method is proposed in this article.

3. Materials and Methods

3.1. Pervasive Auto-Scaling Method (PASM) for Computing Resource Allocation (CRA)

The design goal of the manuscript is to verify the quality of resource allocation for performing multiple application services to provide adaptable load balancing at different application traffic intervals using deep reinforcement learning. The proposed PASM for CRA is to improve the application quality of services. The PASM processes two concepts, namely the auto-scaling concept and migration concept, performed from the current cloud resource provider. Figure 1 presents the diagrammatic illustration of the proposed method.

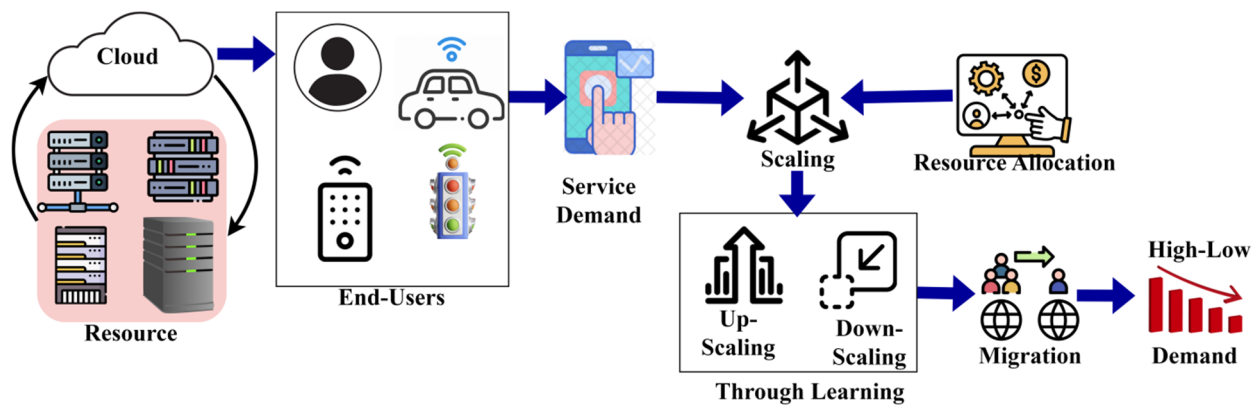


Figure 1. Diagrammatic Representation of PASM-CRA.

The Auto-scaling concept is used to give adaptable load balancing in the varying application traffic intervals. In this concept, the learning network is used to accurately classify and verify up-scaling and down-scaling sharing instances. These instances are used for identifying the overflow in application demands based on service failures and wait time. The wait time and failures are aided by training the learning network. Deep reinforcement learning employs the computation of resource allocation without failure and waiting time. The high-quality resources are allocated using the auto-scaling method through the proposed method for migrating resource providers based on the high/low application demands. The service demands serve as the input for verifying up-scaling and down-scaling output. The up-scaling and down-scaling sharing instances output is used to identify the overflowing application demands. The service demands are serving inputs to the scaling process, which involves computing resource allocation for their service demand ratio. The PASM is used to analyze the application service demands via a learning network to improve the quality of services. Both auto-scaling and cloud computing are balanced to maximize computing resource allocation that augments application service quality. The scaling is decided based on the resource allocation to the application demand ratio by the learning network according to the successive computing intervals. The service demands serve as the input of an auto-scaling method to avoid service failures and wait time.

The concept of “pervasive” pertains to the Pervasive Auto-Scaling Method (PASM) for Computing Resource Allocation (CRA) in cloud environments. This approach offers flexible load balancing in response to fluctuating application traffic intervals and consistently checks the sharing of instances for scaling up and down over different time frames. PASM enhances the quality of service for applications by improving resource distribution and allows the system to continuously evaluate and adapt to evolving application needs and service demands. It supports the transition of resource providers based on the ratio of high to low demand between consecutive computing intervals and promotes ongoing training of the learning network to optimize resource distribution, thereby minimizing wait times and service disruptions. The pervasive characteristic of this method ensures that auto-scaling and resource allocation are continuous, adaptive processes that cater to the dynamic requirements of the system, enabling repeated processes that optimize resource allocation with high sharing ratios.

3.2. Learning Implication

The actual aim of this learning is to accurately verify the up-scaling and down-scaling sharing sequences based on cloud resource providers. The learning network is trained using scaling rates from the previous (completed) allocation intervals, and outputs are processed to address the overflowing application demands of the resource allocation using

the learning network. The auto-scaling method is achieved by computing the sensed and actuated inputs and outputs of robots. The training-assisted adaptable load balancing is performed to verify the scaling rate at which the failed computing interval is identified. For instance, the end-user service demands are used as inputs.

$$E_{Us}(c_t + 1) = Load_{bal}(Appl_t + Qos) - (UP_{scale} + Down_{scale}) \quad (1)$$

where

$$V(UP_{scale} + Down_{scale}) = Load_{bal}(Appl_t + Qos) - (RA_t) \quad (2)$$

Equations (1) and (2) demonstrate flexible load balancing for various application intervals and confirm the results of scaling up and down at different resource allocation time intervals. As per Equations (1) and (2), the variables $Load_{bal}(Appl_t + Qos)$ and $Vo(UP_{scale} + Down_{scale})$ represent the adaptable load balancing for varying application intervals, and verification output of up-scaling and down-scaling is observed at different computing resource allocation time intervals. Where RA represents the resource allocation, and the time for resource allocation RA_t are observed during the quality-of-service analysis.

3.3. Quality of Service Analysis

In this quality-of-service analysis, deep reinforcement learning is used to compute the overflow in application demands for their service failure and wait time. The actual process of cloud computing identifies the application demands for the auto-scaling method output at different application demand intervals. A maximum CRA is identified in any application demand intervals to improve the quality of service QoS of the cloud resources. The main aim of reinforcement learning is to avoid wait time. Hence, the QoS of resource allocation, which is computed from the auto-scaling method output $scaling_O$.

$$QoS(scaling_O) = Appl_{demand} - Ov_f \quad (3)$$

Here, $Appl_{demand}$ and Ov_f are the overall application demand intervals and overflowing demands are computed using the reinforcement learning process. Equation (3) addresses the overflow in application demands, tackling service failure and wait time using deep reinforcement learning. The demands and services are computed from the available cloud resources that are used for the following resource allocation through learning. The learning-based quality resource allocation is performed to avoid service failures between the up-scaling and down-scaling sharing instances in cloud platforms. For instance, the overflowing application demands are addressed to improve the CRA to the demand ratio that results in either 1 or 0. The scaling process flow is represented in Figure 2.

The initial interval allocation process relies on the $\frac{Appl_t}{Service\ providers}$ until $E_{us} > RA$ is true. If this condition is true, then overflow is experienced, after which the available service providers (SP) are identified. The condition $RA = RA_t$ expresses the overflow suppression by sharing E_{us} to the RA_t with new SP . Therefore, the scaling is demanding if $RA \neq RA_t$ for up and down processes. If $RA > RA_t$, upscaling is required; $RA < RA_t$ demands downscaling (Figure 2).

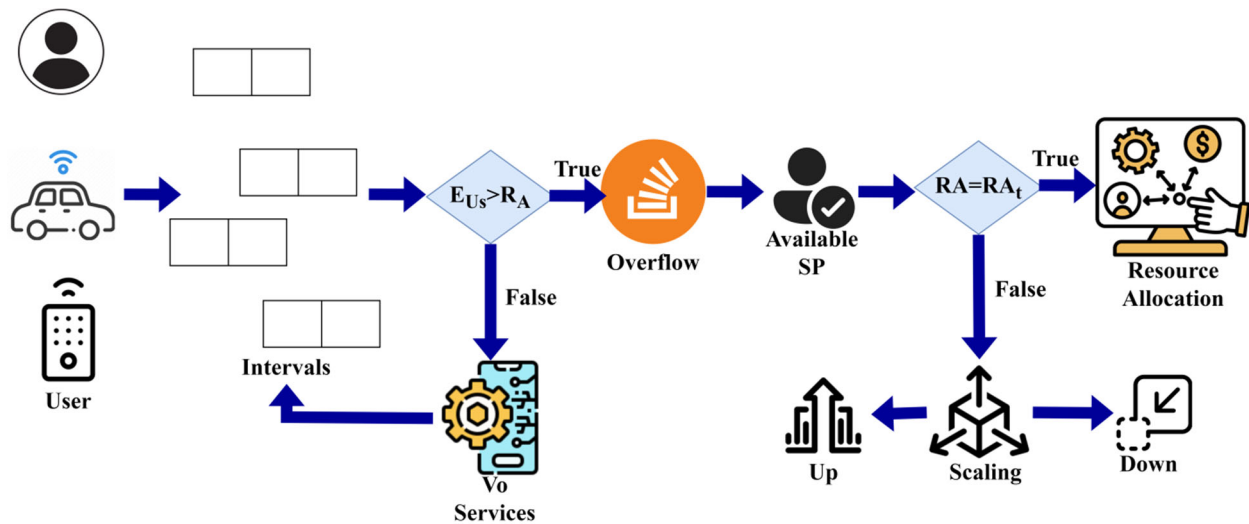


Figure 2. Scaling Process Flow Representation.

3.4. Identifying the Overflowing Application Demands

In this step, the application demands observed from the cloud platform are used for identifying overflow. The application demands are observed from the users based on their services in the cloud platform and then computing overflow. This computing interval is utilized to identify the wait time and service failures in cloud computing through learning. Therefore, the $Appl_{demand}$ and Of_f at a random location for the varying application traffic intervals is given as

$$Appl_{demand} = \frac{Wt}{Wt + Sf} \quad (4)$$

where the variables Wt and Sf represent wait time and service failures detected from cloud computing.

$$Of_f = \frac{RA_f}{RA_f + Sdy} \quad (5)$$

Here, RA_f and Sdy is defined as resource allocation failure, and service delay is detected for accurate computation of scaling rate with the following CRA. Equations (4) and (5) assess wait time, service failures, resource allocation failure, and service delay to precisely determine the scaling rate.

3.5. Auto-Scaling Method

The Auto-Scaling method is a prominent consideration here for the proposed method. The main role of the proposed method is to verify the up-scaling and down-scaling ratios based on the available application demands and the services observed from the cloud resources. Different application demands are observed from the random locations, which helps to train the learning process for maximizing resource allocation with adaptable load balancing and previous resource allocation under varying application traffic intervals. For instance, the appropriate training and computing resource allocation are performed to reduce service delays and failures. This computation generates the number of users, number of application demands, scaling rate, number of resource allocations, number of users waiting for resource allocations, and number of services in the learning network. The reinforcement learning provides less allocation wait time, service delay, allocation failures, and resource utilization, thereby maximizing resource allocation. The learning process

provides high computing resource allocation. Based on the demands and services, the final scaling rate is given as

$$scaling_{rate} = 2Load_{bal}(Appl_t + Qos)X_1 - Ov_f \quad (6)$$

and

$$Prev_{CAi} = 2X_2 + \frac{QoS(scaling_O)}{Appl_{demand} + Ov_f} \quad (7)$$

Here, the variable X_1 , X_2 and $Prev_{CAi}$ represent the two random variables and previously completed allocation intervals contain the associated cloud resource information at random locations, regardless of the application demand intervals. Equation (6) illustrates the verification output associated with the processes of up-scaling and down-scaling observed at various time intervals for computing resource distribution. The function used integrates these two variables to generate the verification output $scaling_{rate}$. This output likely acts as a metric or indicator of the effectiveness of resource up-scaling and down-scaling in relation to the allocation time intervals. This equation is instrumental in assessing the efficiency and effectiveness of dynamic resource allocation in computing systems, especially in scenarios where quality of service is a crucial consideration. It facilitates the evaluation of how well the system adjusts its resource allocation over time to satisfy service demands or performance criteria. The auto-scaling process decision is illustrated in Figure 3.

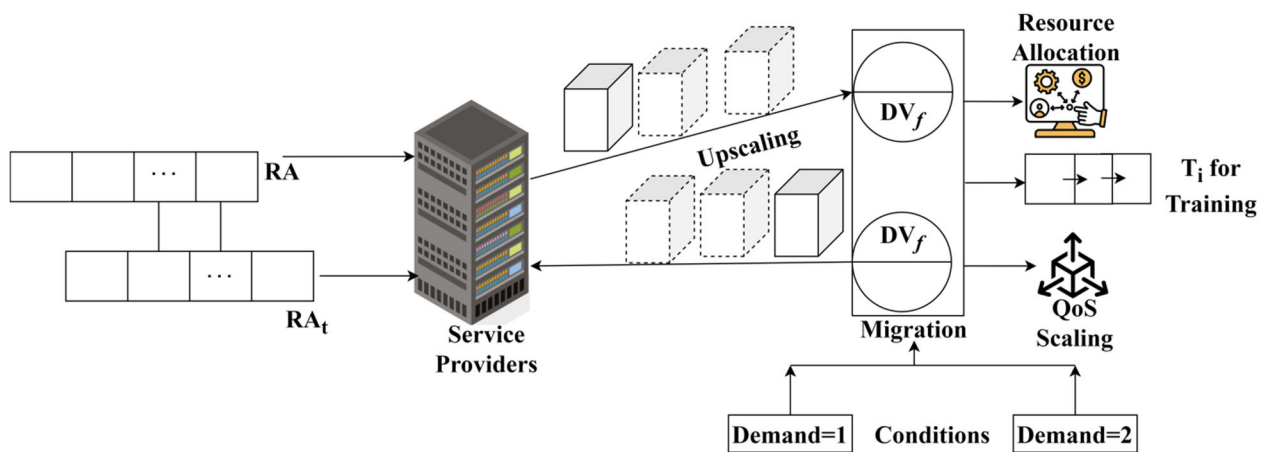


Figure 3. Auto-Scaling Process Decision.

The auto-scaling process illustration is presented in Figure 3. This auto-scaling relies on RA and RA_t for the different service providers (allocated and unallocated). If both the allocations are concurrent, for upscaling under $Appl_{Demand} = 1$ and downscaling for $Appl_{Demand} = 0$. This refers to $(RA + RA_t)$ demands for upscaling to reduce wait time and therefore the migration for OV_f is pursued. The alternate case of RA_t is alone used for downscaling, which reduces service failures. This demands Qos-based scaling where migration is optimal for fewer scaling rates.

3.6. Training the Learning Network

The appropriate and accurate resource allocation is performed based on training the learning network and the auto-scaling method output, without overflowing application demands, which is a reliable output. The overflow of application demands is identified and reduced to improve the quality of resource allocation with the computed scaling rate pursued using deep reinforcement learning, respectively. The scaling output is supported by all the computing resource allocation intervals at different application demand intervals,

for which the wait time and service failure are reduced. The scaling verification is performed to achieve the maximum computing resource allocation at random locations. In CRA, the multiple services of $Serv_1$ which are processed and $Serv_2, Serv_3$ are also processed to avoid further service failures during the training process to improve the quality of resource allocation in cloud computing. All the resources are arranged as a priority and trained based on scaling output for achieving a high sharing ratio. Both demands and services are observed to prevent service failures and delays in cloud computing. That observation-based resource allocation with scaling output is performed to reduce service failures, delays, and wait time, also maximizing the demand and sharing ratio based on migrating resource providers. This process is recurrent until maximum resource allocation with a high sharing ratio is the optimal output. The resource allocation rate differs based on the scaling output, which uses the high quality of services for maximizing the resource allocation rate for random locations and application traffic intervals. The proposed PASM is designed to train the learning network to reduce service, demand delays, and failures. Hence, the scaling output is performed by migrating the resource providers depending on the training and computing resource allocation. This row of resource allocation RA_{sqc} sequence with a high sharing ratio is expressed as

$$RA_{sqc}(t+1) = \frac{Appl_{t_1}.Prev_{CAi_1} + Appl_{t_2}.Prev_{CAi_2} + \dots + Appl_{t_n}.Prev_{CAi_n}}{Ov_f} \quad (8)$$

$$\left. \begin{aligned} RA_1 &= |T_1 - Prev_{CAi_1} \cdot \Delta_\alpha| \\ RA_2 &= |T_2 - Prev_{CAi_2} \cdot \Delta_\beta| \\ &\vdots \\ RA_n &= |T_i - Prev_{CAi_n} \cdot \Delta_\gamma| \end{aligned} \right\} \quad (9)$$

As per Equation (8), T_1, T_2 and T_i are the recurrent training of learning networks under varying application traffic intervals with fewer service failures. Equation (8) explains the ongoing training of learning networks under different application traffic intervals with minimized service failures. For which the scaling rate using learning network training is pursued from the previous (completed) allocation intervals. $Prev_{CAi_1}, Prev_{CAi_2}$ and $Prev_{CAi_n}$ are computed from the completed resource allocation time intervals based on the migration of resource providers for a high-to-low demand ratio. $\Delta_\alpha, \Delta_\beta$ and Δ_γ are the service and demand computed from the current resource allocation intervals, using neural network learning. Equation (8) (non-linear) describes a recurrent training process for a learning network with varying application traffic intervals. The equation involves several variables and operations, indicating a complex interplay between different elements in the auto-scaling method. Without additional context or specific details about the variables and their interconnections, it is challenging to definitively determine the linearity or non-linearity of this equation. It appears to be part of a more intricate system that includes resource allocation, scaling rates, and application demands within a cloud computing environment. The network training process is illustrated in Figure 4.

The learning process relies on two training instances: $(t-1)$ and $(t+1)$ for Sf and Wt reduction. The $Scaling_{rate}$ (up/down) are the inputs for $(RA + RA_t$ and $RA)$ are verified for CA_{i_n} to identify T_i for $Appl_{demand}$. If the demand is validated, then RA for different n is observed to reach “ sqc ”. If this output is different from $(t-1)$ and $(t+1)$, the migration is performed. Such migration mismatch validates the need for recurrent training under various RA . Therefore, the learning network’s training is inducted until sfc is reduced from $(t+1)$ or increased from $(t-1)$ (Figure 4).

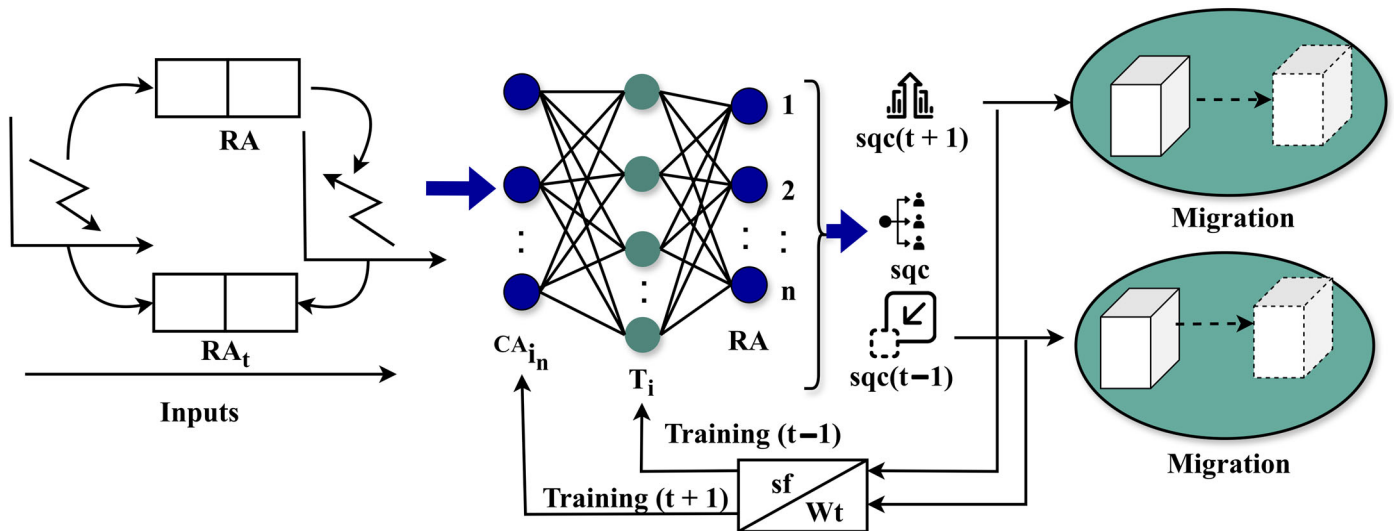


Figure 4. Network Training Process.

3.7. Resource Allocation

The learning network identifies the service failures and delays in any cloud resources. Based on the instance, the scaling is decided to generate a successive allocation interval that performs the appropriate resource provider migration. An auto-scaling method is proposed to achieve high-quality resource allocation in the cloud platform. Based on the method, up-scaling and down-scaling are prominent in the following resource utilization and allocation processes that are used for improving Qos. The service failures and delays are reduced to better meet application demands from the cloud users. To eliminate such issues, this manuscript proposes that training-assisted CRA is pursued based on application demands and its services through learning.

First, the application demands are observed from the cloud users through the available devices or resources. Maximum resource allocation with sharing accuracy is achieved, and the overflowing application demands are reduced to prevent service delays and failures. The proposed method used to compute resource allocation with service failure instances is recurrently trained between the up-scaling and down-scaling to achieve a maximum sharing ratio. Here, the application demands and services an input via scaling verification, which is computed for resource allocation.

3.8. Migration Process

The actual aim of the proposed method is to achieve high resource utilization and allocation depending on the high-to-low demand ratio between successive computing intervals using a reinforcement learning process by migration of resource providers to reduce wait time. This computation enhances the resource allocation based on the recurrent training and scaling rate. This process aims to reduce the number of service demand failures and wait time. Therefore, the migration of resource providers is expressed as

$$Migrate_{ResP}\left(\frac{Demd_{high}}{Demd_{low}}\right) = \frac{\mathbb{C}(UP_{scale} \rightarrow Down_{scale}) + RA_{Demd_{high}}}{\mathbb{C}(UP_{scale} \leftarrow Down_{scale}) + RA_{Demd_{low}}} \quad (10)$$

In Equation (10), $Migrate_{ResP}\left(\frac{Demd_{high}}{Demd_{low}}\right)$ used to represent the resource provider is migrated based on the demand ratio between computing intervals. Computing the application demand ratio for achieving high-quality resource allocation with sharing is to improve the overall performance. Equation (10) details the migration of resource providers according to the demand ratio between computing intervals. The conditions of $\mathbb{C}(UP_{scale} \rightarrow Down_{scale})$

and $\mathcal{C}(UP_{scale} \leftarrow Down_{scale})$ shows the migration of up-scaling to down-scaling and migration of down-scaling to up-scaling are pursued by a learning network for maximizing resource allocation in cloud resources. The accurate demands and services of the cloud users are observed to allocate resources with a high sharing ratio. Based on the high-to-low demand ratio, the failure and wait time addressed resource providers are migrated to their location between successive computing intervals. The service provider migration process is given in Figure 5. This migration process is different for up-scaling and down-scaling resources.

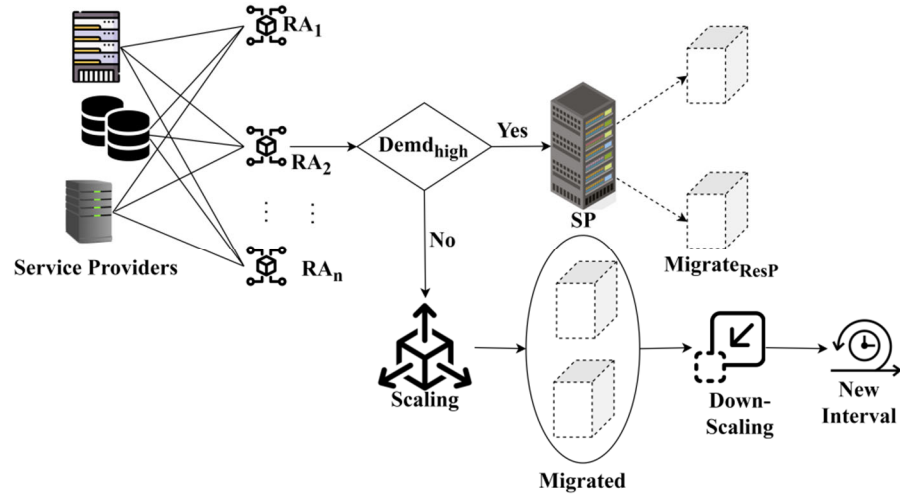


Figure 5. Migration Process for Up and Down-Scaling.

The service providers are validated to ensure maximum RA_n to satisfy the service demands. Across the allocation instances, the WT and Sf are the T_i constraints that are to be satisfied. Therefore if $Demd_{high}$ persists apart from the scaling process, and then $Migrate_{Resp}$ is performed. If the demand is under less consent, then the previously migrated resources are scaled to a new interval. Such down-scaled resources are migrated for $Demd_{high}$ user services (Figure 5).

3.9. Allocation Wait Time and Failure Detection

The recurrent process is performed to reduce the wait time and operation costs of the application demand intervals and maximize resource allocation. It is maximized by reducing the wait time, failures, and operation costs regardless of the application demand intervals. It also reduces the wait time during the migration process until maximum resource allocation with high sharing is achieved. The high quality of services provides maximum resource utilization and allocation from the learning network.

$$Wt = \frac{(Demd_{high} - Demd_{low})}{Ov_f} + Serv_{min} \quad (11)$$

and,

$$Sf = 1/\sqrt{2\pi} \left[\frac{\left(\mathcal{C}(UP_{scale} \rightarrow Down_{scale}) - \frac{Serv}{Ov_f} \right)}{2(Cur_{CAi_n} - \emptyset_{CAi_n})} \right] \quad (12)$$

In this, the wait time and service failure between the current computing allocation interval Cur_{CAi_n} and the successive computing allocation interval $\emptyset_{CAi_n}$ is to achieve maximum resource allocation in the cloud platform. Equations (11) and (12) compute wait time and service failure between the current and next computing allocation intervals

to enhance resource allocation. The proposed PASM is used to ensure the quality of resource allocation with recurrent training and scaling verification, assisted by a high-to-low demand ratio, which reduces wait time and failure. The optimal output is less wait time and operation costs to perform multiple services in a cloud platform using scaling and migration, which is fed to the learning process for achieving high-quality services with less wait time. The proposed method is described in Algorithm 1.

Algorithm 1. Proposed PSAM-CRA.

Initialize cloud_environment with service_providers and users

Step 1: For each application_demand_interval:

 Observe user_service_demands

 Compute overflow in application_demands

 Identify wait_time and service_failures

Step 2: Train learning_network:

 Use scaling_rates from previous allocation_intervals

 Verify up_scaling and down_scaling sharing_instances

 Compute resource_allocation_to_demand_ratio

Step 3: Perform auto_scaling:

 If overflow is detected:

 Adjust scaling

 Compute new resource_allocation

 Update learning_network

Step 4: Migrate resource_providers:

 Calculate the high-to-low-demand ratio between intervals

 Migrate providers to reduce wait time

 Update resource_allocation

Step 5: Repeat steps 3–5 until maximum resource allocation with high sharing is achieved

Continuously monitor and adjust for QoS improvement.

The balancing of up-scaling and down-scaling sharing instances enhances the resource allocation rate. The proposed method improves resource allocation with a sharing ratio and thereby reduces wait time and failures. The proposed method is analyzed for self-assessment using the metrics used internally. These metrics are identified as a part of the comparative analysis under the same simulation/experimental environment. Therefore, the parameters such as resource allocation rate, demand estimated, and the resource factors are considered for evaluating the methods. Similarly, the number of service providers involved in the scaling process is also a numerical parameter used in this assessment. The connection between the number of service providers, the allocation factor, and the scaling rate is used to evaluate the following metrics. In the first analysis, the RA and RA_t for different E_{us} is presented.

In the below comparison (Figure 6), the QoS and the application response for different E_{Us} factor is presented. The comparison takes place for the QoS requirement factor and the application response factor independently. This does not consider the QoS and the application; rather, the variables used represent the QoS factor and the application response factor estimated from the maximum requests/demands received in t . The rate of resource allocated is validated for both factors across allocation and response. The RA and RA_t are applicable for down and up-scaling processes, respectively. In this case, the training using $(t - 1)$ and $(t + 1)$ are independent for maximizing allocation. The migration and

$Scaling_{rate}$ are defined at high levels to increase the RA . However, if the learning process identifies $sqc(t+1)$ then migration to balance RA_t is performed. This migration is consented to reduce $sqc(t+1)$ until a better allocation is sustained. Therefore, the RA until n instances follow CA_{in} for which migration is performed alone. For the varying service providers, the $scaling_{Rate}$ analysis is presented in Figure 7 below.

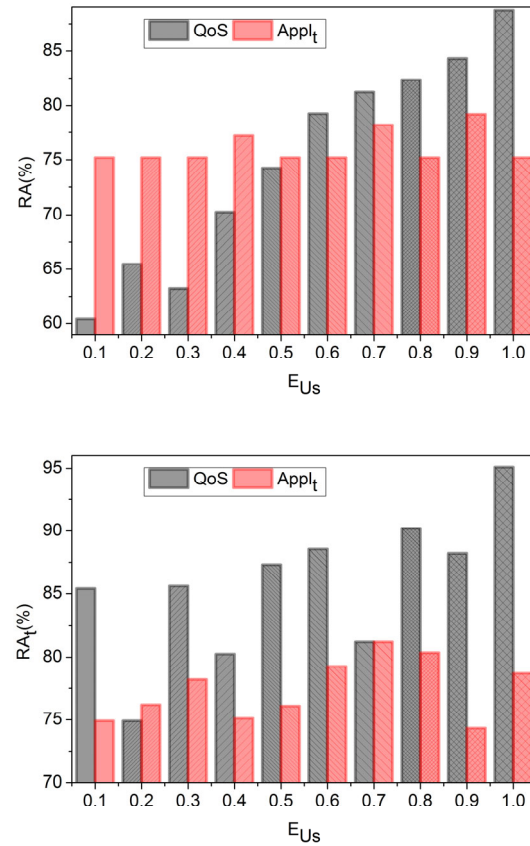


Figure 6. RA and RA_t Analyses.

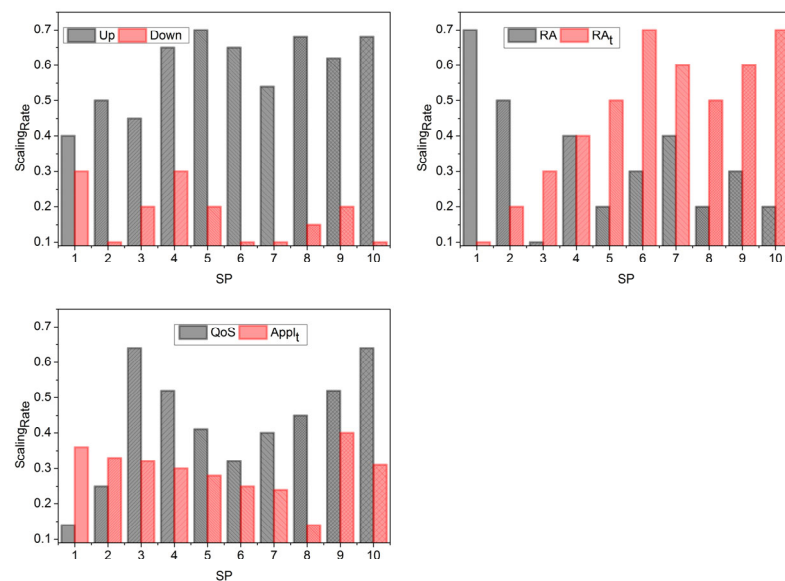


Figure 7. $Scaling_{Rate}$ Analyses.

The $Scaling_{Rate}$ is analyzed under (up, downscaling), (RA_t, RA) and (QoS, $Appl_t$) for the varying service providers. The learning process identifies $E_{us} > RA$ and $RA = R_t$

condition satisfaction is overflow and migration assessment. In the distinguishable learning process, the T_i and CA_{i_n} differentiation is mandatory. Based on these conditions and constraints, the QoS and $Appl_t$ based scaling is induced for $(t + 1)$ and $(t - 1)$ sequences. Therefore, the scaling down process is pursued for distinguishable intervals. This enhances the SP migration and scaling across various QoS demands (Figure 7). The Sf for the $(t + 1)$, t and $(t - 1)$ sequences are analyzed under $Scaling_{Rate}$ and T_i iterations in Figure 8.

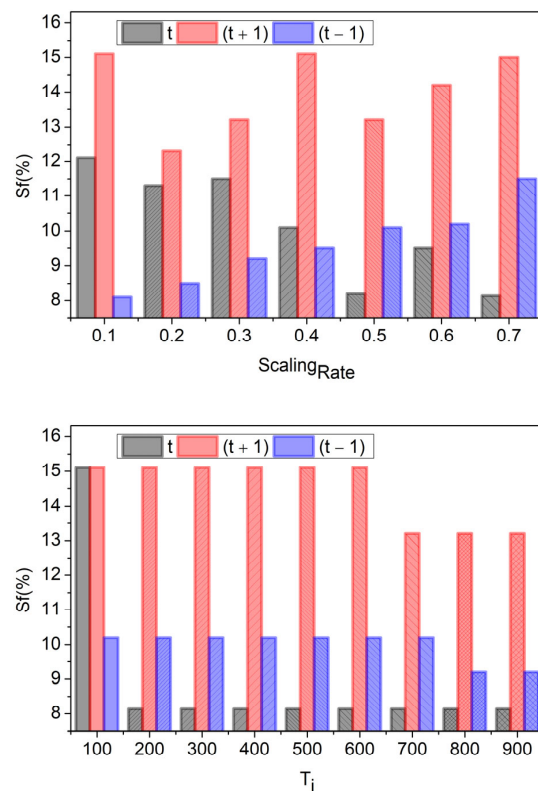


Figure 8. Sf Analyses for $Scaling_{Rate}$ and T_i .

The scaling rate varies the allocation under RA and RA_t to reduce the failures through classified learning. The $Migrate_{ResP}$ and seq_c are the corresponding factors to increase resource allocation. In this allocation, RA and RA_t are consented to leverage the allocation regardless of the demand. Therefore, the new T_i is validated for $seq_c(t - 1)$ and $seq_c(t + 1)$ to reduce the Sf . The proposed method is reliable across different SP and allocation intervals.

4. Results and Discussion

The results and discussion section is provided with metric-based analysis extracted from an OPNET-based simulation. The simulation environment is provided with an open cloud platform with 10 service providers and 50+ users. The services are shared under a minimum interval of 30 s and a maximum interval of 360 s, depending on the user demand. The learning network is modeled with 900 iterations for sf and WI training. The learning rate is set between 0.5 and 1.0, targeting 20 epochs at any time. In addition, the decay rate is 0.2 with a pause time of 2 s between each iteration. Using this simulation environment, the resource allocation rate, service delay, wait time, allocation failure, and resource utilization metrics are comparatively analyzed. In this comparative analysis, the proposed method is accompanied by ADA-RP [29], PA-RF [20], and EAS-VMM [25] methods. The learning network undergoes repeated training until it achieves optimal resource allocation with significant sharing, indicating a process of iterative enhancement. This method seeks to manage dynamic workloads by examining application requirements, confirming scaling

instances, pinpointing excessive demands, and relocating resource providers according to demand ratios. The proposed method is designed to support scalability based on varying service providers and intervals. Both these variants impose on the user count, increasing the application demands processed per unit interval. Therefore, the number of changes identified across different intervals maximizes the resource allocation by confining the downtime. This ensures a flexible changeover to the different resource allocation functions by scaling up and scaling down resources. Therefore, scalability is nevertheless an issue with this proposed method.

4.1. Resource Allocation Rate

In Figure 9, the proposed method maximizes the resource allocation rate in the cloud platform for preventing service failures and wait time. The auto-scaling method is used to verify that up-scaling and down-scaling sharing instances under varying application traffic intervals are verified and then provide adaptable load balancing to improve the application quality of service. The CRA at random locations is computed using deep reinforcement learning for the demand ratio. The overflowing application demands are identified for their service failures. Those overflow-identified instances are used to train the learning network based on the sharing ratio. The application demands observed from multiple users are analyzed for maximizing resource allocation and minimizing service failures. Cloud computing is performed to identify and reduce service failures and wait time based on the high-to-low demand ratio between successive computing intervals. The up-scaling and down-scaling are verified through the learning process for improving the application demands in the cloud platform. The learning is trained to employ two scaling processes designed for maximizing the resource allocation rate.

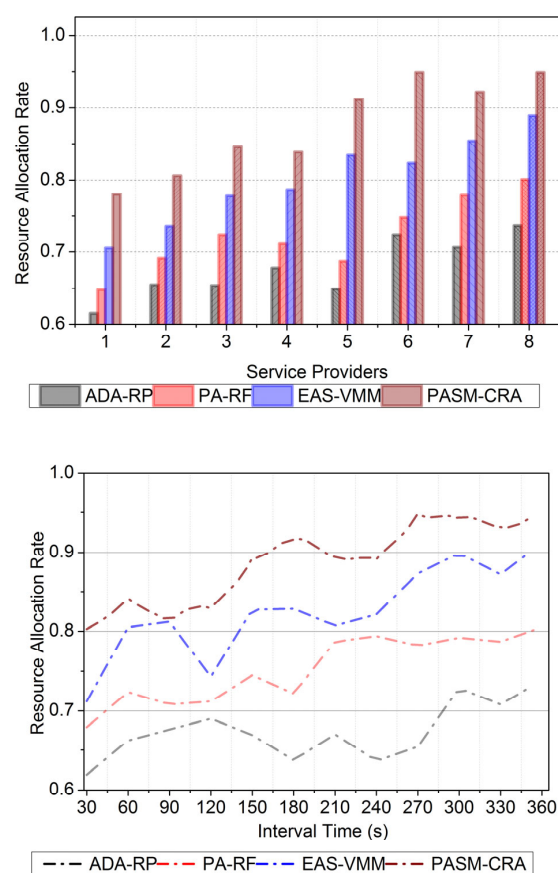


Figure 9. Resource Allocation Rate.

4.2. Service Delay

The quality of service-based computing resource allocation is pursued using deep reinforcement learning for achieving failure-free application demand and services through learning. This is being done to reduce service delays as represented in Figure 10. Learning is used to improve the quality of services and thereby reduce failures and wait time. The learning method is employed to decide the up-scaling and down-scaling from the previously completed allocation intervals. The proposed method results in fewer service delays based on a high-to-low demand ratio using the scaling rate generated by the learning process. In this auto-scaling concept, the learning network is used to accurately verify up-scaling and down-scaling sharing instances based on their application demands. These instances are used for identifying the overflowing application demands in cloud computing due to service failures, delays, and wait time. The wait time and failures identified resources are used to train the learning network to maximize resource allocation. The service failure is addressed using a scaling process for providing adaptable load balancing. Both scaling methods are designed as inversely proportional to each other for their service failure. Hence, the high-to-low demand ratio between the successive computing intervals is to minimize service delay.

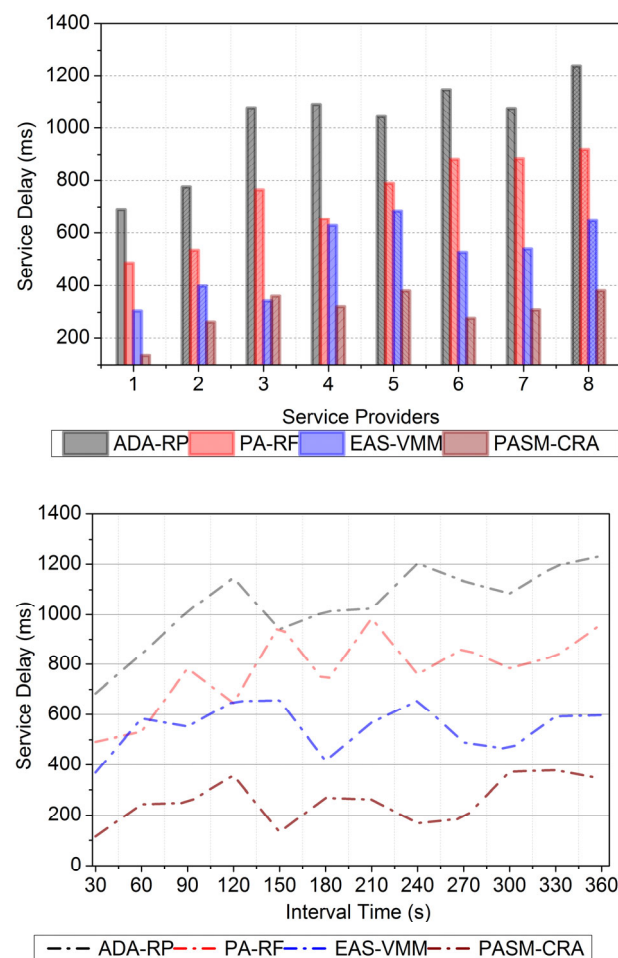


Figure 10. Service Delay.

4.3. Wait Time

The wait time observed from the available resources is analyzed for reducing service failures in cloud computing through learning, as illustrated in Figure 11. This analysis is performed to identify up-scaling and down-scaling of the services for varying applica-

tion demand intervals. In this proposed method, deep reinforcement learning employs computing resource allocation without failure, and the waiting time is the optimal output. The high-quality resources are allocated using the auto-scaling method. For instance, the resource provider migrates based on the high-to-low application demands for achieving successive computing intervals. Regardless of the application demand intervals, both scaling-based sharing instances are used, for which the service delay and wait time are mitigated through learning. In this article, the service failures and wait time are mitigated by the demand ratio through the proposed method and learning. Based on the overflowing application demands, the linear output of $Appl_{demand}$ and $Overf$ is to maximize the learning training. Based on the demand and services, the learning-based high-quality resources are allocated to avoid service failure and wait time.

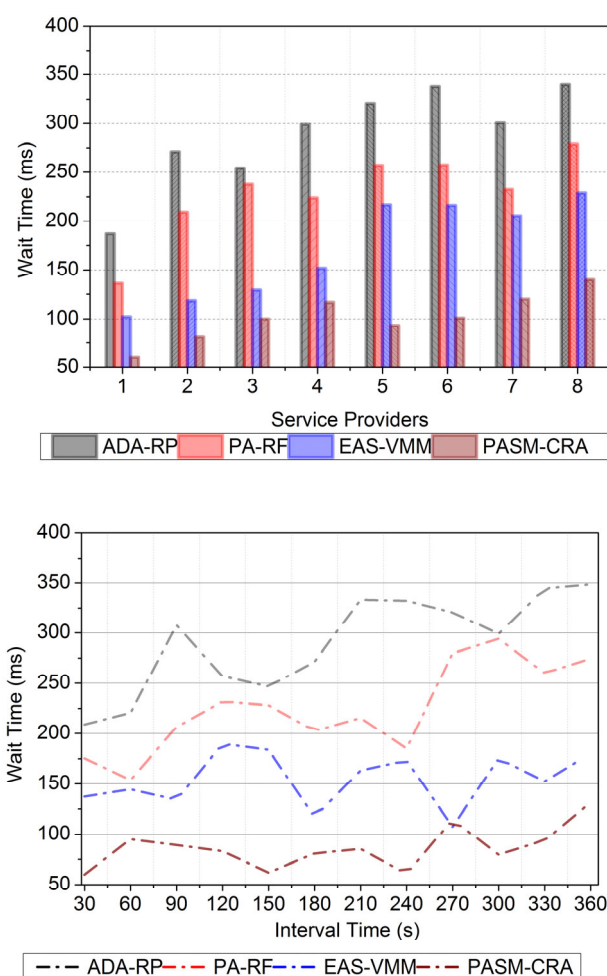


Figure 11. Wait Time.

4.4. Allocation Failure

In this proposed auto-scaling method, less wait time and service failures are observed to achieve maximum resource allocation. This is based on their services in the cloud platform, and addressing the overflow application demands is represented in Figure 12. Addressed service and allocation failures are detected from the sequential resource allocation intervals for reducing wait times in cloud platforms. The overflowing application demands increase the operation costs and wait time for all the resources through learning. The computing resource allocation is performed to provide better adaptable load balancing. In this article, the allocation failure and service delay are reduced when validating the overflowing application demands at random locations. Reducing such allocation failures in

the sequential resource allocation is pursued to improve the quality of resource allocation with fewer service failures and wait time in the cloud platform. For instance, learning is used to decide on scaling. Different application demands are observed from the random locations, which helps to train the learning process to maximize resource allocation with adaptable load balancing and previous resource allocation intervals. The learning network is trained using scaling rates from the completed allocation intervals to reduce allocation failure.

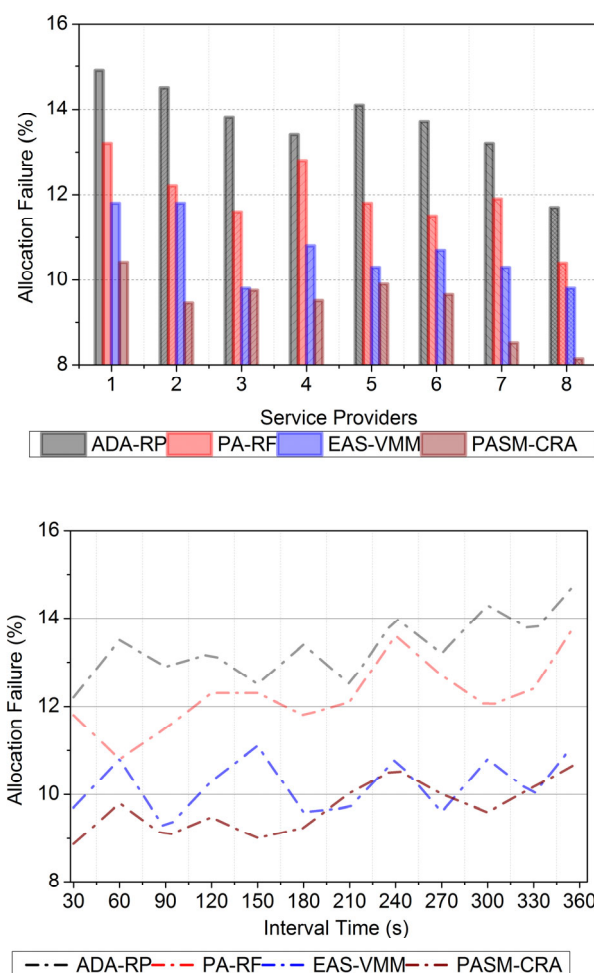


Figure 12. Allocation Failure.

4.5. Resource Utilization

The proposed method improves the resource allocation for the application demand ratio (Refer to Figure 13). This process is recurrently pursued until the maximum resource allocation with a high sharing ratio. For this sequential training of the learning network and scaling rate used to satisfy high resource utilization with less wait time and service failure, the successive computing interval is used. That interval is focused on preventing allocation failures through learning. The overflowing application demands are identified and reduced based on the scaling rate and high-to-low demand ratio. This process is performed to improve the quality of resource allocation with fewer failures, using deep reinforcement learning. Both demands and services are observed from the current and previous computing allocation intervals to reduce service failures and delays in cloud computing. That observation-based resource allocation with quality of services is performed to reduce service failures and delays and thereby maximizes the demand and sharing ratio. Based on the resource providers that are migrated, the demand ratio is accurately identified between

successive allocation intervals. Using the proposed method and learning is to satisfy high resource utilization with fewer failures in this manuscript. The results discussed above are tabulated in Tables 1 and 2.

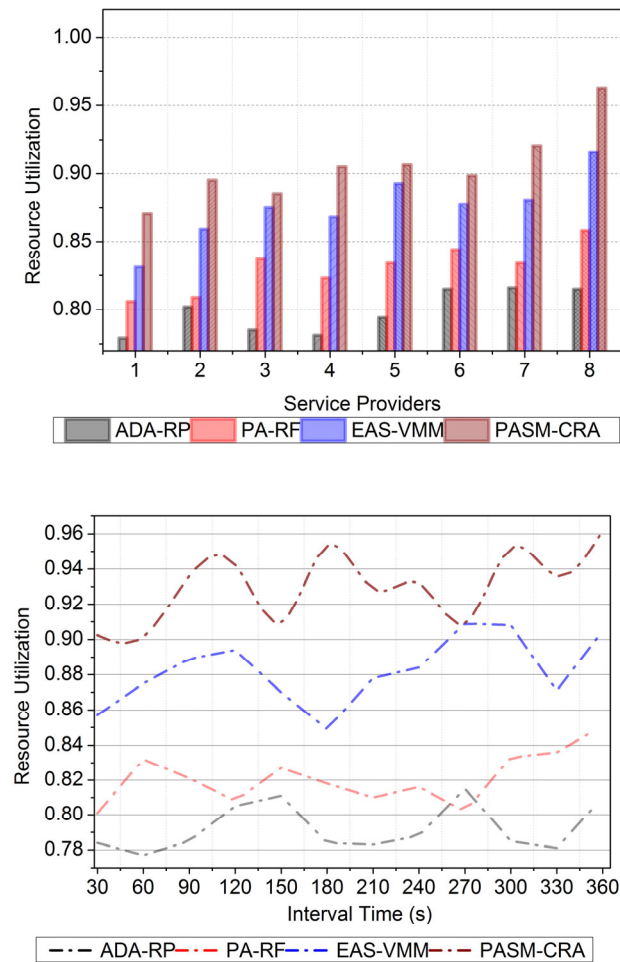


Figure 13. Resource Utilization.

Table 1. Results for Service Providers.

Metrics	ADA-RP	PA-RF	EAS-VMM	PASM-CRA
Resource Allocation Rate	0.737	0.801	0.889	0.9487
Service Delay (ms)	1236.9	919.2	646.1	380.33
Wait Time (ms)	339.6	278.8	228.6	139.87
Allocation Failure (%)	11.7	10.4	9.8	8.14
Resource Utilization	0.815	0.858	0.916	0.9629

The proposed PSAM-CRA improves the resource allocation rate by 13.97% and resource utilization by 9.99%; it reduces the service delay by 9.88%, wait time by 8.41%, and allocation failure by 7.48% for the different service providers.

Table 2. Results for Interval Time.

Metrics	ADA-RP	PA-RF	EAS-VMM	PASM-CRA
Resource Allocation Rate	0.739	0.805	0.912	0.9539
Service Delay (ms)	1232.5	960.8	596.3	344.16
Wait Time (ms)	348.5	274.3	180.3	131.51
Allocation Failure (%)	14.9	14.02	11.3	10.74
Resource Utilization	0.81	0.851	0.905	0.9619

The proposed PSAM-CRA improves the resource allocation rate by 13.52% and resource utilization by 10.66%; it reduces the service delay by 10.5%, wait time by 8.48%, and allocation failure by 8% for the different service providers.

5. Conclusions

This article proposes a pervasive auto-scaling method for computing resource allocation in the cloud. This method was designed to improve the quality of service based on resource allocation in the cloud with user demand considerations. This method incorporated auto-scaling, including the up- and down-scaling processes, assisted by deep reinforcement learning. The allocation demands and the completed intervals are accounted for in training the learning network for new allocations and demand suppressions. This recurrent process is validated to reduce the wait time through scaling and migration based on overflow demands and high sharing rates. The process exploited the scaling rates to ensure high resource sharing, from which high-to-low service failures were observed. Therefore, the learning network was trained independently from this perspective to increase the service failure sessions regardless of the pending demands. From the experimental analysis, the proposed PSAM-CRA improves the resource allocation rate by 13.97% and resource utilization by 9.99%; it also reduces the service delay by 9.88%, wait time by 8.41% and allocation failure by 7.48% for the different service providers.

Author Contributions: Conceptualization, V.R.R. and G.S.; Methodology, G.S. and V.R.R.; software, G.S.; validation, V.R.R. and G.S.; formal analysis, G.S.; investigation, V.R.R.; resources, G.S.; data curation, V.R.R.; writing—original draft preparation, V.R.R. and G.S.; writing—review and editing, V.R.R.; visualization, G.S.; supervision, G.S.; project administration, V.R.R.; funding acquisition, V.R.R. and G.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: Not applicable.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Dogani, J.; Khunjush, F.; Mahmoudi, M.R.; Seydali, M. Multivariate workload and resource prediction in cloud computing using CNN and GRU by attention mechanism. *J. Supercomput.* **2023**, *79*, 3437–3470. [\[CrossRef\]](#)
2. Kashyap, S.; Singh, A. Prediction-based scheduling techniques for cloud data center's workload: A systematic review. *Clust. Comput.* **2023**, *26*, 3209–3235. [\[CrossRef\]](#)
3. Park, E.; Baek, K.; Cho, E.; Ko, I.Y. Fully Decentralized Horizontal Autoscaling for Burst of Load in Fog Computing. *J. Web Eng.* **2023**, *22*, 849–870. [\[CrossRef\]](#)

4. Jeong, B.; Jeon, J.; Jeong, Y.S. Proactive Resource Autoscaling Scheme based on SCINet for High-performance Cloud Computing. *IEEE Trans. Cloud Comput.* **2023**, *11*, 3497–3509. [\[CrossRef\]](#)
5. Dogani, J.; Khunjush, F.; Seydali, M. K-AGRUED: A Container Autoscaling Technique for Cloud-based Web Applications in Kubernetes Using Attention-based GRU Encoder-Decoder. *J. Grid Comput.* **2022**, *20*, 40. [\[CrossRef\]](#)
6. Wu, C.; Sreekanti, V.; Hellerstein, J.M. Autoscaling tiered cloud storage in Anna. *Proc. VLDB Endow.* **2019**, *12*, 624–638. [\[CrossRef\]](#)
7. Stupar, I.; Huljenic, D. Model-based cloud service deployment optimisation method for minimisation of application service operational cost. *J. Cloud Comput.* **2023**, *12*, 23. [\[CrossRef\]](#)
8. Rajasekar, P.; Palanichamy, Y. Adaptive resource provisioning and scheduling algorithm for scientific workflows on IaaS cloud. *SN Comput. Sci.* **2021**, *2*, 456. [\[CrossRef\]](#)
9. Sahni, J.; Vidyarthi, D.P. Heterogeneity-aware elastic scaling of streaming applications on cloud platforms. *J. Supercomput.* **2021**, *77*, 10512–10539. [\[CrossRef\]](#)
10. Rabiou, S.; Yong, C.H.; Mohamad, S.M.S. A Cloud-Based Container Microservices: A Review on Load-Balancing and Auto-Scaling Issues. *Int. J. Data Sci.* **2022**, *3*, 80–92. [\[CrossRef\]](#)
11. Verma, V.K.; Gautam, P. Evaluations of Distributed Computing on Auto-Scaling and Load Balancing Aspects in Cloud Systems. *Int. J. Appl. Math. Comput. Sci. Syst. Eng.* **2020**, *2*, 31–37.
12. Psychas, K.; Ghaderi, J. A Theory of Auto-Scaling for Resource Reservation in Cloud Services. *ACM SIGMETRICS Perform. Eval. Rev.* **2021**, *48*, 27–32. [\[CrossRef\]](#)
13. Fé, I.; Matos, R.; Dantas, J.; Melo, C.; Nguyen, T.A.; Min, D.; Choi, E.; Silva, F.A.; Maciel, P.R.M. Performance-Cost Trade-Off in Auto-Scaling Mechanisms for Cloud Computing. *Sensors* **2022**, *22*, 1221. [\[CrossRef\]](#)
14. Abdullah, M.; Iqbal, W.; Mahmood, A.; Bukhari, F.; Erradi, A. Predictive autoscaling of microservices hosted in fog microdata center. *IEEE Syst. J.* **2020**, *15*, 1275–1286. [\[CrossRef\]](#)
15. Jazayeri, F.; Shahidinejad, A.; Ghobaei-Arani, M. Autonomous computation offloading and auto-scaling the in the mobile fog computing: A deep reinforcement learning-based approach. *J. Ambient Intell. Humaniz. Comput.* **2021**, *12*, 8265–8284. [\[CrossRef\]](#)
16. kumar Kandru, A.; Sharma, N. Energy Efficient Resource Management In Cloud Computing By Laod Balancing And Auto Scaling. *Turk. J. Comput. Math. Educ. (TURCOMAT)* **2020**, *11*, 423–431.
17. Khaleq, A.A.; Ra, I. Intelligent autoscaling of microservices in the cloud for real-time applications. *IEEE Access* **2021**, *9*, 35464–35476. [\[CrossRef\]](#)
18. Desmoucheaux, Y.; Enguehard, M.; Clausen, T.H. Joint monitorless load-balancing and autoscaling for zero-wait-time in data centers. *IEEE Trans. Netw. Serv. Manag.* **2020**, *18*, 672–686. [\[CrossRef\]](#)
19. Iqbal, W.; Erradi, A.; Abdullah, M.; Mahmood, A. Predictive auto-scaling of multi-tier applications using performance varying cloud resources. *IEEE Trans. Cloud Comput.* **2019**, *10*, 595–607. [\[CrossRef\]](#)
20. Al Qassem, L.M.; Stouraitis, T.; Damiani, E.; Elfadel, I.A.M. Proactive Random-Forest Autoscaler for Microservice Resource Allocation. *IEEE Access* **2023**, *11*, 2570–2585. [\[CrossRef\]](#)
21. Zeydan, E.; Mangues-Bafalluy, J.; Baranda, J.; Martínez, R.; Vettori, L. A multi-criteria decision making approach for scaling and placement of virtual network functions. *J. Netw. Syst. Manag.* **2022**, *30*, 32. [\[CrossRef\]](#)
22. Bento, A.; Araujo, F.; Barbosa, R. Cost-Availability Aware Scaling: Towards Optimal Scaling of Cloud Services. *J. Grid Comput.* **2023**, *21*, 80. [\[CrossRef\]](#)
23. Jena, T.; Mohanty, J.R.; Satapathy, S.C. Categorization of intercloud users and auto-scaling of resources. *Evol. Intell.* **2021**, *14*, 369–379. [\[CrossRef\]](#)
24. Feng, X.; Ma, J.; Liu, S.; Miao, Y.; Liu, X. Auto-scalable and fault-tolerant load balancing mechanism for cloud computing based on the proof-of-work election. *Sci. China Inf. Sci.* **2022**, *65*, 112102. [\[CrossRef\]](#)
25. Verma, S.; Bala, A. Efficient Auto-scaling for Host Load Prediction through VM migration in Cloud. *Concurr. Comput. Pract. Exp.* **2024**, *36*, e7925. [\[CrossRef\]](#)
26. Rout, S.K.; Ravinda, J.V.R.; Meda, A.; Mohanty, S.N.; Kavididevi, V. A Dynamic Scalable Auto-Scaling Model as a Load Balancer in the Cloud Computing Environment. *EAI Endorsed Trans. Scalable Inf. Syst.* **2023**, *10*, 1–7. [\[CrossRef\]](#)
27. Sharvani, G.S. An auto-scaling approach to load balance dynamic workloads for cloud systems. *Turk. J. Comput. Math. Educ. (TURCOMAT)* **2021**, *12*, 515–531. [\[CrossRef\]](#)
28. Llorens-Carrodegua, A.; Leyva-Pupo, I.; Cervelló-Pastor, C.; Piñeiro, L.; Siddiqui, S. An SDN-based solution for horizontal auto-scaling and load balancing of transparent VNF clusters. *Sensors* **2021**, *21*, 8283. [\[CrossRef\]](#) [\[PubMed\]](#)
29. Chouliaras, S.; Sotiriadis, S. An adaptive auto-scaling framework for cloud resource provisioning. *Future Gener. Comput. Syst.* **2023**, *148*, 173–183. [\[CrossRef\]](#)

30. Kim, I.K.; Wang, W.; Qi, Y.; Humphrey, M. Forecasting cloud application workloads with cloudinsight for predictive resource management. *IEEE Trans. Cloud Comput.* **2020**, *10*, 1848–1863. [[CrossRef](#)]
31. Adewojo, A.A.; Bass, J.M. A novel weight-assignment load balancing algorithm for cloud applications. *SN Comput. Sci.* **2023**, *4*, 270. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.